



Human Interface Devices

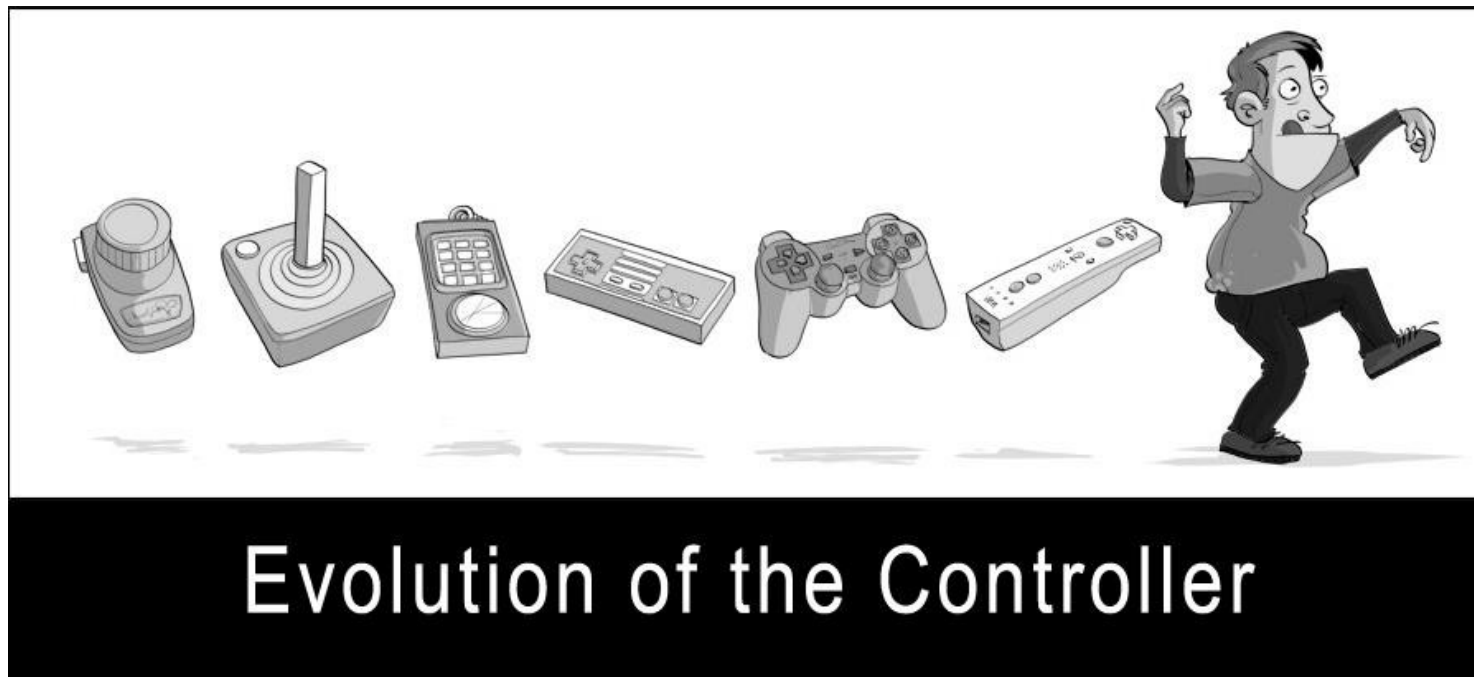
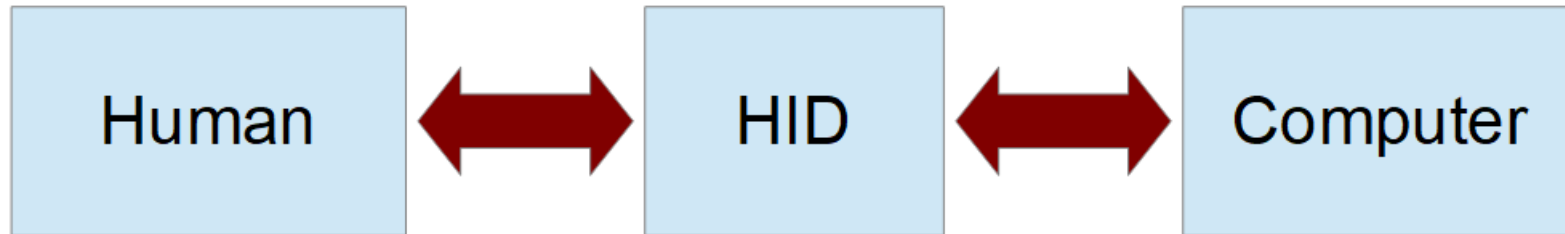


Image sources: <http://www.metallman.com/2012/03/evolution-of-video-game-controller.html>



- A **Human Interface Device (HID)** is a type of computer device that interacts directly with, and most often takes input from, humans and may deliver output to humans.



HID Examples (1/3)



- Ordinary HID Devices



Image sources: https://brain-images-ssl.cdn.dixons.com/2/6/21426662/u_21426662.jpg, https://www.vesalia.de/e_competitionusb.htm, <https://de.wikipedia.org/wiki/Datei:Atari-2600-Joystick.jpg>, <https://www.estarland.com/product-description/NintendoNES/NES-Controller-by-Nintendo/14492>, https://en.wikipedia.org/wiki/Nintendo_64_controller, <https://www.windowcentral.com/titan-one-controller-accessory>

HID Examples (2/3)



- Weird HID Devices



Image sources: <https://commons.wikimedia.org/wiki/File:Virtual-Boy-Controller.jpg>, <https://en.wikipedia.org/wiki/File:Nintendo-SNES-Super-Scope-L.jpg>, <http://fiftyfootant.com/way-to-go-genius/nnes-powerglove/>, <https://consolepassion.co.uk/platform/philips-cdi?page=1>, <https://en.wikipedia.org/wiki/File:Atari-5200-Controller.jpg>, <https://en.wikipedia.org/wiki/File:Atari-Jaguar-Controller.jpg>, <http://nintendo.wikia.com/wiki/U-Force>

HID Examples (3/3)



- Fancy HID Devices



Image sources: <http://us.playstation.com/ps3/accessories/playstation-move-charging-station-ps3.html>, <https://phys.org/news/2013-03-microsoft-version-kinect-windows-sdk.html>, https://www.researchgate.net/publication/51451999_Anger_and_the_Speed_of_Full-Body_Approach_and_Avoidance_Reactions/figures?lo=1, <http://www.mixdownmag.com.au/evolution-guitar-hero-controllers>, <https://www.talkandroid.com/guides/misc/guide-use-your-wiimote-to-play-emulators/>

Interfacing with a HID



- Game software reads and writes HID inputs and outputs in various ways, depending on the specific design of the device in question.
 - Depends on the platform and on the used HID device
 - PC / Home console / handheld
 - Wired / wireless / built-in
 - Input only / input + output
 - Getting the Device State
 - Input data (Buttons, analog-sticks, triggers, ...)
 - Additional information (Connection state, extension devices, error codes, ...)
 - Writing to the device
 - Rumble / LEDs / audio / video
 - Device memory (Wiimote Miis, memory card, ...)
 - Many game engines provide high-level APIs
 - Unified device handling and input mapping
 - Normalized input data

Polling



- Querying the state of the device once per game loop iteration
 - Used for Simple devices (game pads, old-school joysticks)
 - Reading input
 - Reading hardware registers directly, reading a memory-mapped I/O port, or via a higher-level software interface
 - Writing output
 - Writing to special registers or memory-mapped I/O addresses, or via a higher-level API
 - Characteristics
 - Minimal amount of source code needed
 - Application is required to query frequently enough
 - APIs are typically unique to the hardware
 - Examples
 - Microsoft XInput API → `XInputGetState()`
 - Nintendo Dolphin SDK → `PADRead()`



Image sources: <https://de.wikipedia.org/wiki/Datei:Atari-2600-Joystick.jpg>

Interrupts / Callbacks / Messages



- Some devices communicates with the host via hardware interrupts
 - An **interrupt** is an electronic signal generated by the hardware, which causes the CPU to temporarily suspend execution of the main program
 - In the case of a HID, the interrupt handler code will probably read the state of the device and store it off for later processing
 - The game engine can pick up the data the next time it is convenient to do so
 - Typical for mouse and keyboard
 - There's no reason to send a continuous stream of data between the mouse and the computer when the mouse isn't moving
 - Device callbacks can be registered
 - Called when the state of the device changes
 - Examples
 - GLUT → `glutMouseFunc()`, `glutKeyboardFunc()`, ...
 - Windows API → `LRESULT CALLBACK WndProc(...)`

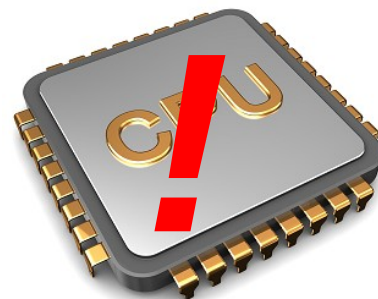


Image sources: https://askleo.com/why_wont_my_program_use_more_than_25_of_the_cpu/

Wireless Devices



- The inputs and outputs of a wireless (e.g. Bluetooth) device cannot be read and written by simply accessing registers or memory-mapped I/O ports
 - PS5, Xbox, Switch, ...
 - The software must “talk” to the device via the Bluetooth protocol
 - The software can request the HID to send input data
 - The software can send output data
 - Often handled by a thread separate from the game engine’s main loop
 - Additional error handling is needed
 - Connection problems
 - Transmission delays
 - Low battery power
 - From the point of view of the game programmer, the state can be made to look pretty much indistinguishable from a traditional polled device
 - Characteristics are then very similar to polling
 - E.g. Nintendo Revolution SDK → `WPADRead()` / `KPADRead()`



Image sources: https://en.wikipedia.org/wiki/List_of_Bluetooth_profiles

Types of Input



- Although human interface devices for games vary widely in terms of form factor and layout, most of the inputs they provide fall into one of a small number of categories
 - Digital values
 - Analog values (absolute or relative)
 - Complex data
 - Cameras (images or video)
 - Microphone (frequencies and loudness)
- Special HIDs may act like regular controllers, e.g.
 - Guitar Hero Controller → Gamepad
 - Dance Mat → Gamepad
 - Touch Screen → Mouse Cursor
 - ...

Digital Buttons (1/2)



- Almost every HID has at least a few digital buttons
 - Two states: *pressed* and *not pressed*
 - The state of a digital button is usually represented by a single bit
 - 0 → not pressed
 - 1 → pressed
 - The states of all buttons can be packed into a single unsigned integer value



```
#define          BUTTON_UP          0x0001
#define          BUTTON_DOWN        0x0002
#define          BUTTON_LEFT        0x0004
/* ... */
unsigned short uiButtonState = readButtonInput();

if    (uiButtonState & BUTTON_UP)      { /* ... */ }

else if (uiButtonState & BUTTON_DOWN)  { /* ... */ }
```

Image sources: <https://www.estarland.com/product-description/NintendoNES/NES-Controller-by-Nintendo/14492>

Digital Buttons (2/2)



- Derivation of transition states

```
unsigned short    uiButtonState;  
unsigned short    uiButtonStateOld;  
unsigned short    uiButtonTrigger;  
unsigned short    uiButtonRelease;  
  
while (true)      /* main loop */  
{  
    uiButtonStateOld = uiButtonState;  
    uiButtonState    = readButtonInput();  
  
    uiButtonTrigger  = ( uiButtonStateOld ^ uiButtonState ) & uiButtonState;  
  
    uiButtonRelease  = ( uiButtonStateOld ^ uiButtonState ) & uiButtonStateOld;  
    /* ... */  
}
```

Analog Axes and Buttons (1/4)



- An *analog input* is one that can take on a range of values
 - Usually digitized, meaning it is quantized and represented using an integer in software, e.g.
 - char $\rightarrow 0 \dots 255$
 - short $\rightarrow -32,768 \dots 32,767$
 - Analog inputs are sometimes called *analog axes* (or just *axes*)
 - Trigger button $\rightarrow$ one axis
 - Analog stick $\rightarrow$ x-axis and y-axis
 - Input values can be *absolute* or *relative*, e.g.
 - Analog stick $\rightarrow$ absolute (zero values if the stick is not tilted)
 - Mouse $\rightarrow$ relative (non-zero values represent a movement delta)



Image sources: Nintendo Wii SDK Documentation

Analog Axes and Buttons (2/4)



- Normalizing input

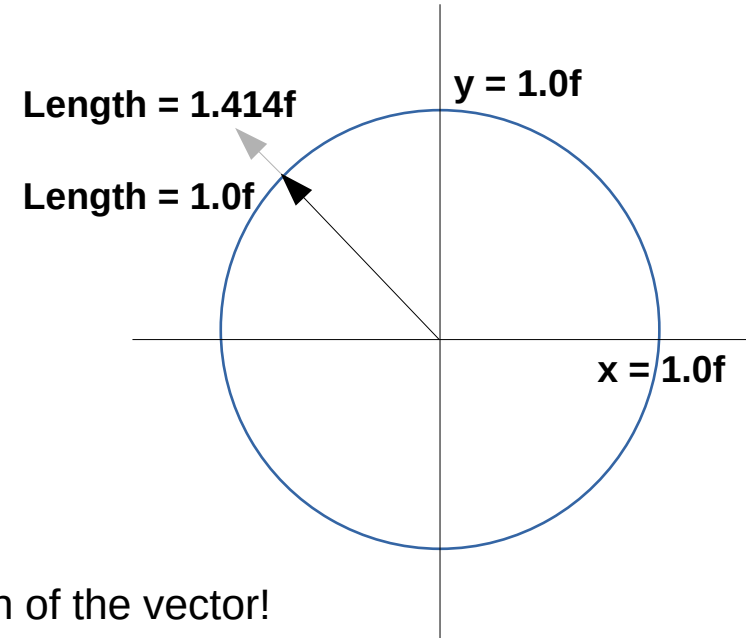
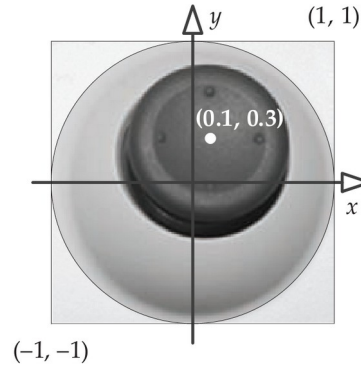
- $0 \dots 255 \rightarrow 0 \dots 1$
- $-32,768 \dots 32,767 \rightarrow -1 \dots 1$

$$\Rightarrow n = v / m$$

n: normalized input value

v: raw input value

m: maximum value



- Character movement

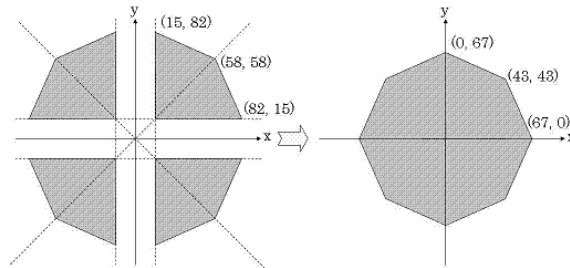
- The movement speed needs to be mapped to the length of the vector!
- The same is true for digital movement (WASD)

Image sources: <http://www.spyparty.com/2010/08/30/have-an-alpha-masked-xbox-360-controller-on-me/comment-page-1/>

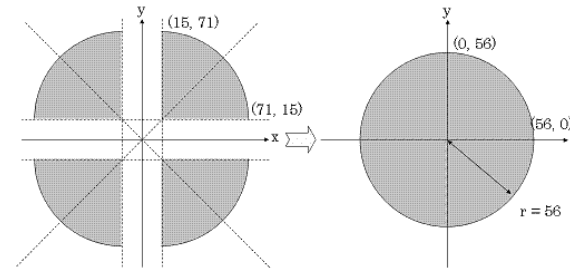
Analog Axes and Buttons (3/4)



- Because HID devices are analog devices, the voltage produced is noisy, and the actual inputs we observe may fluctuate slightly
- A *dead zone* is any area of a control interface that has no input effect.
 - Any input values that are within the *dead zone* are simply clamped.
 - The *dead zone* must be wide enough to account for the noisiest inputs generated by an undisturbed control, but small enough not to interfere with the player's sense of the HID's responsiveness.
 - Example: Nunchuk Thumbstick



WPADClampStick(FREESTYLE)
WPAD_STICK_CLAMP_OCTA_WITH_PLAY



WPADClampStick(FREESTYLE)
WPAD_STICK_CLAMP_CIRCLE_WITH_PLAY

Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

Analog Axes and Buttons (4/4)



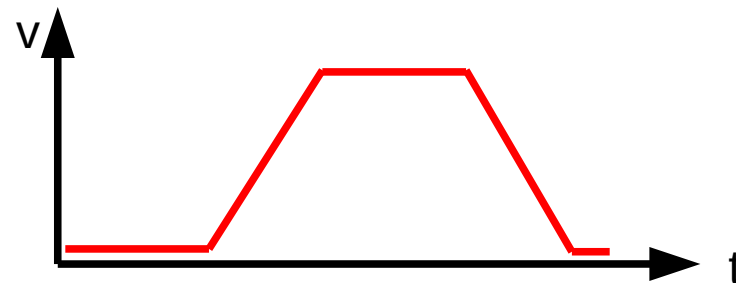
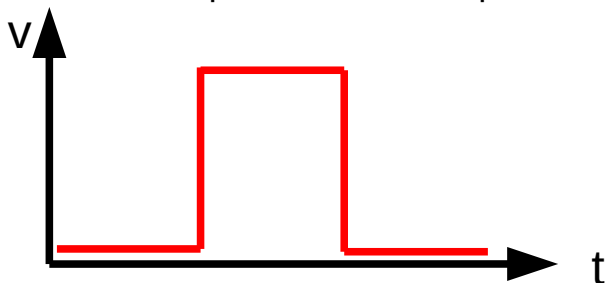
- *Ramping* is another way of saying *accelerating*
 - Using raw input values can make movement seem very jarring
 - “How fast can you tilt an analog stick back and forth?”

// Ramp the acceleration by the elapsed time.

```
float numberOfSeconds = 2.0f;
```

```
m_currentSpeed +=  
    m_maxSpeed * ( (elapsedTime*elapsedTime) / numberOfSeconds);
```

```
if (m_currentSpeed > m_maxSpeed)  
    m_currentSpeed = m_maxSpeed;
```



Accelerometers (1/2)



- *Accelerometers* can detect *acceleration* along the three axes x, y, z
 - Used in Wiimote, Nunchuk, PS3/4/5 Controller, PS Move, Smartphones, ...
 - These are *relative* analog inputs (like a mouse's two-dimensional axes)
 - When the controller is not accelerating these inputs are zero

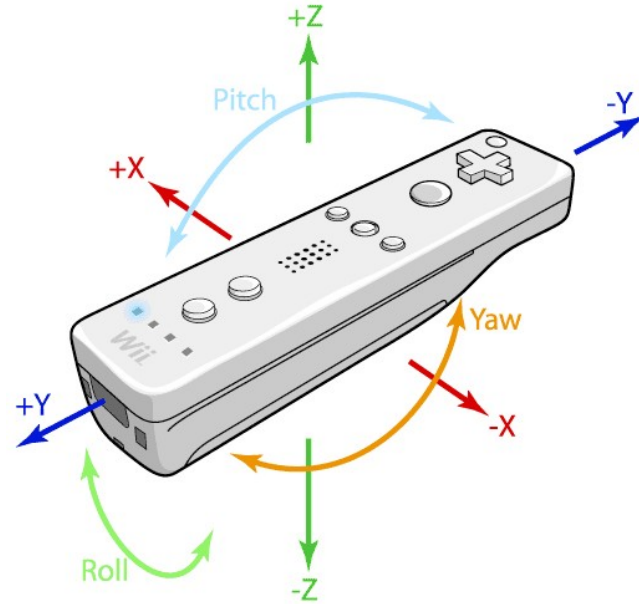
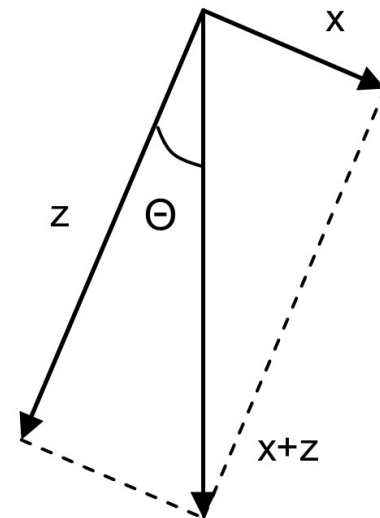


Image sources: <https://osculator.net/doc/faq:wiimote>

Accelerometers (2/2)



- On Earth there is a constant downward acceleration due to gravity
 $\Rightarrow 1g \approx 9.8 \text{ m/s}^2$
 - If the controller is held perfectly level, the vertical acceleration should be approximately $-1g$
 - The orientation of the controller can be estimated
 - Zero position needs to be calibrated initially
 - Cannot be accurately measured for very fast movements because the acceleration due to the force of movement is greater than the force of gravity due to orientation
 - Cannot be used for calculating gravity and orientation in the horizontal direction (rotational angle in the horizontal direction)
 - It is almost impossible to determine the absolute position of the controller
 - Value range is limited (e.g. $\pm 3.0g$)
 - Influence of gravity
 - Inaccuracies / tolerance



$$\tan \Theta = x/z$$
$$\Theta = \arctan(x/z)$$

Image sources: Nintendo Wii SDK Documentation

Gyroscopes



- Gyroscopes can measure *angular velocity* in the pitch, yaw, and roll directions
 - Used in Wii MotionPlus, PlayStation Move, Switch Joy Cons, ...
 - *Relative* analog inputs
 - When the controller is not rotating the inputs are zero
 - Used in combination with accelerometers
 - Can measure angular velocity independent of gravity
 - Can lead to a better estimation of position and orientation
 - Inaccuracies / tolerances are still an issue



Image sources: https://www.researchgate.net/publication/266007911_FoleySonic_Placing_Sounds_on_a_Timeline_Through_Gestures/figures?lo=1, <https://en.wikipedia.org/wiki/Gyroscope>

Magnetometers



- A *magnetometer* is used to measure the strength and/or direction of a magnetic field
 - Can be used for calibrating a controller's orientation against the Earth's magnetic field
 - This helps to correct against cumulative error (drift) by other inertial sensors like accelerometers or gyroscopes
 - In combination with accelerometers, the controller's 3D orientation can be calculated
 - Used in PlayStation Move, Wii U Controller, Smartphones, ...
 - Problems
 - There's a wide variety of different sensors
 - Output values tend to drift (e.g. due to changes in temperature)

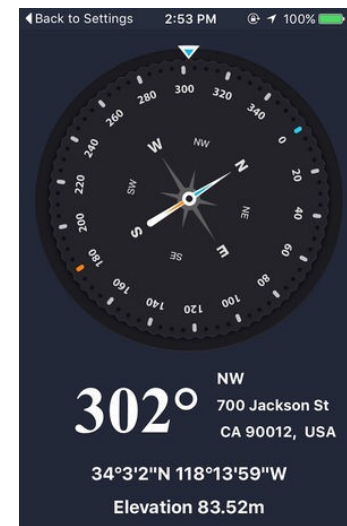


Image sources: <https://www.soutdoors.com/best-compass-app/>, <https://de.wikipedia.org/wiki/Datei:Sony-PlayStation-Move-Controller.png>

Cameras (1/2)



- **Cameras** may be used as input devices for games
 - Cameras deliver a sequence of images to the computer
 - EyeToy, PlayStation Eye, Kinect, Smartphone, Joy Con IR, etc.
- Possibilities
 - Detecting the player and their movement
 - Using your hands or your entire body as a controller
 - Tracking of objects or light sources
 - Motion controllers, markers, VR headsets, etc.
 - Also can provide a reference point for tracking
 - Input of RGB video or image data
 - Textures for in-game objects, video recordings, etc.
 - Augmented Reality
 - Camera shows reality and the game adds elements on top
 - Face recognition and detection of emotions
 - IR cameras can even measure a person's heart beat

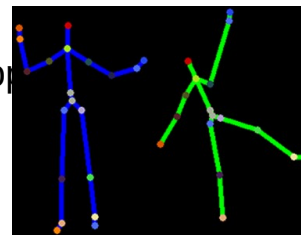


Image sources: <https://en.wikipedia.org/wiki/EyeToy>, <https://www.mobgames.com/game/ps2/eyetoy-play-2/promo/promoImageId,314041/>, <https://www.youtube.com/watch?v=22YoHk0Yi2I>, <http://www.youtube.com/watch?v=Hi5kMNfgDS4>, <https://phys.org/news/2013-03-microsoft-version-kinect-windows-sdk.html>

Cameras (2/2)



- Downsides of using cameras for input
 - Lots of processing power required for image/video processing
 - E.g. Xbox 360 dedicated one entire processing core to the Kinect
 - Limited field of view → only works, if the player is in front of the camera
 - Objects can easily get occluded (hand behind back, etc.)
 - Potential latency issues, especially at lower frame rates
 - Resolution might be too low and details can get lost
 - Detection problems (especially motions and gestures)
 - No haptic feedback for the player (touching the air)
 - Dependency on the lighting in the room (too bright or too dark)
 - For tracking, at least some object details are necessary in the environment
 - Foreign light sources may be mistaken for tracking points (e.g. candles)

Cameras – Examples (1/2)



- **PlayStation Move** uses the PlayStation Eye to track the wand's position in three dimensions through a special illuminated orb at the end
 - High-contrast colors need to be selected for the orbs
 - Distance and position can be tracked using the size of the light dots
 - Must be re-calibrated due to changing light conditions
- **PlayStation 4/5 Camera** are the successors to the PlayStation Eye
 - Features two cameras (stereoscopic) for better depth sensing
 - Used for tracking the PlayStation VR headset
 - Supports some motion tracking (e.g. player body)
- **Kinect**
 - Features RGB, depth sensor, IR
 - Similar tech is now used in the HoloLens

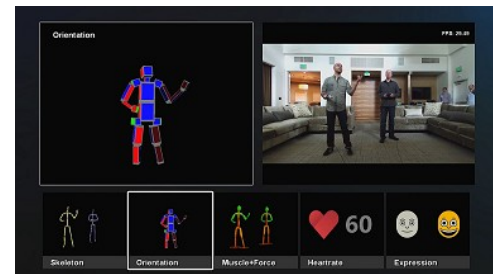


Image sources: <https://www.youtube.com/watch?v=22YoHk0Yi2I>, <https://www.kisspng.com/png-playstation-4-playstation-3-sony-video-game-consol-115310/>

Cameras – Examples (2/2)



- Infrared cameras
 - **Nintendo Wiimote**
 - Records a two-dimensional infra-red image
 - Sensor Bar with two infra-red light sources located at the TV
 - Size and position of the dots is used to calculate the coordinates of the location to which the Wiimote is pointing, the twist angle, and the distance to the Sensor Bar
 - www.youtube.com/watch?v=Jd3-eiid-Uw
 - **Nintendo Switch Joy Con**

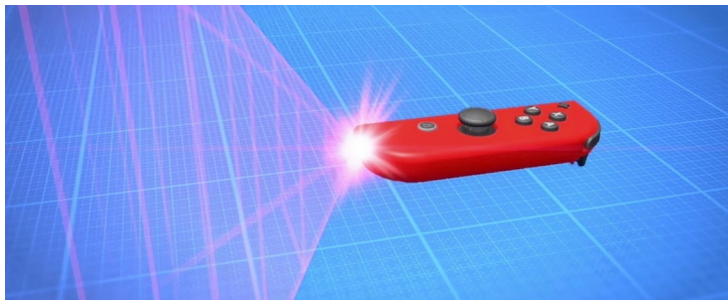
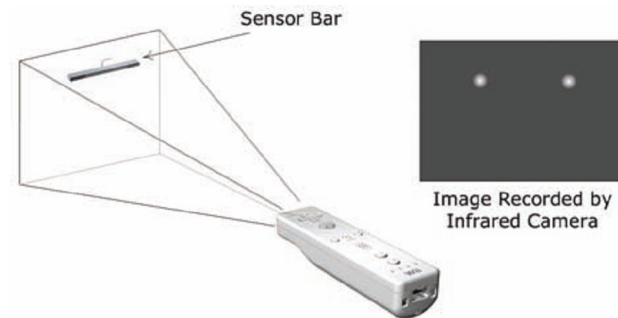


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014, <https://www.nintendo-insider.com/learn-how-the-ir-motion-camera-works-in-nintendo-labos-toy-con-garage/>, <https://labo.nintendo.com/developer-interview-part-2/what-can-you-do-with-the-ir-motion-camera/>

Microphones



- A microphone is an acoustic-to-electric transducer or sensor that converts sound into an electrical signal.



- Applications in video games
 - Voice chat
 - Simple recording and transmission (no data analysis)
 - Noise suppression (filter everything out but the player's voice)
 - Speech recognition
 - Different languages need to be supported separately
 - Singing
 - Frequency analysis (eventually syllable detection)
 - Evaluation of timing, rhythm and speed



Image sources: <https://www.etsy.com/listing/243694492/custom-sound-wave-tattoo-art-voice-art>, <https://www.pinterest.com/pin/382735668329320835/>

Types of Output (1/2)



- Rumble
 - Vibrations are usually produced by one or more motors, each of which rotates a slightly unbalanced weight at various speeds.
 - Can be turned on and off by the game
 - Speed may be controllable as well
- Force-Feedback
 - A technique in which an actuator on the HID is driven by a motor in order to slightly resist the motion the human operator is trying to impart to it.
 - Arcade driving games, flight sticks, ...
- Audio
 - Some HIDs provide outputs that can be utilized by the audio system
 - Built in speakers, headphone jack, ...
- LEDs
 - Indicate connection state, battery life or react to in-game events

Types of Output (2/2)



- Displays



- Head-mounted displays (HMDs)

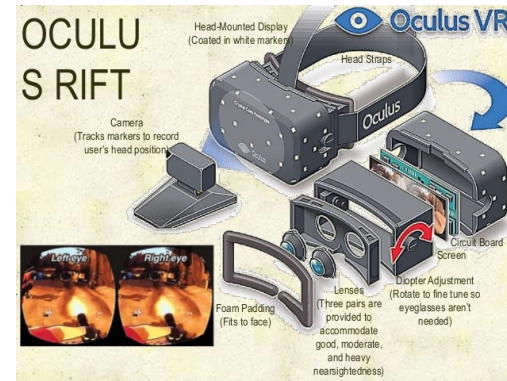


Image sources: <http://www.cashforgamers.com/shop/accessories/dreamcast/official-sega-dreamcast-controller-any-color/>, <https://www.pastemagazine.com/articles/2017/03/8-things-your-new-switch-cant-do-but-the-wii-u-did.html>, https://en.wikipedia.org/wiki/Virtual_Boy, <http://nerdbacon.com/marios-tennis-virtual-boy/>, <https://www.slideshare.net/saurabhsawhney/all-about-virtual-reality-oculus-rift>

Other types of input and output



- Memory cards
- Specialized devices like musical instruments, dance pads, etc.
- Thought-controlled devices
- Holographic projection
- Full-body controllers
- Body-mounted devices
- Infrared sensors
- ...



Image sources: <https://www.tomshardware.com/news/hololens-2-microsoft-mobile-world-congress,38456.html>, <https://hackmag.com/coding/lets-code-for-leap-motion/>, <https://www.techradar.com/news/cybershoes-are-a-visceral-way-to-move-in-vr-at-ces-2019>, <https://www.knoxlabs.com/products/kat-walk-mini-vr-treadmill>

Interlude: Nintendo Switch Joy-Cons



- Clever combination of well-known input technologies
 - Digital buttons
 - Bluetooth 3.0 / Near-Field Communication (NFC)
 - Accelerometer, Gyroscope, Motion IR Camera
 - HD Rumble (weighted electromagnets instead of regular rumble motors?)
 - However... no analog triggers, no microphone, no magnetometer, etc.



Image sources: <https://uae.souq.com/ae-en/nintendo-switch-left-and-right-joy-cons-neon-red-and-neon-blue-22067016/i/>, <https://www.ifixit.com/Teardown/Nintendo+Switch+Teardown/78263>

Interlude: PlayStation 5 and Xbox Series X

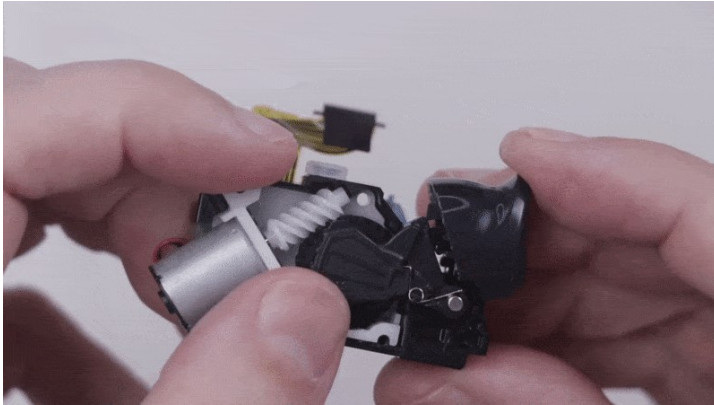


Image sources: <https://www.theverge.com/2020/11/3/21547303/sony-ps5-dualsense-adaptive-triggers-haptics-teardown-features>, <https://news.xbox.com/en-us/2020/03/16/xbox-series-x-controller/>

Runtime Implementation (1/6)

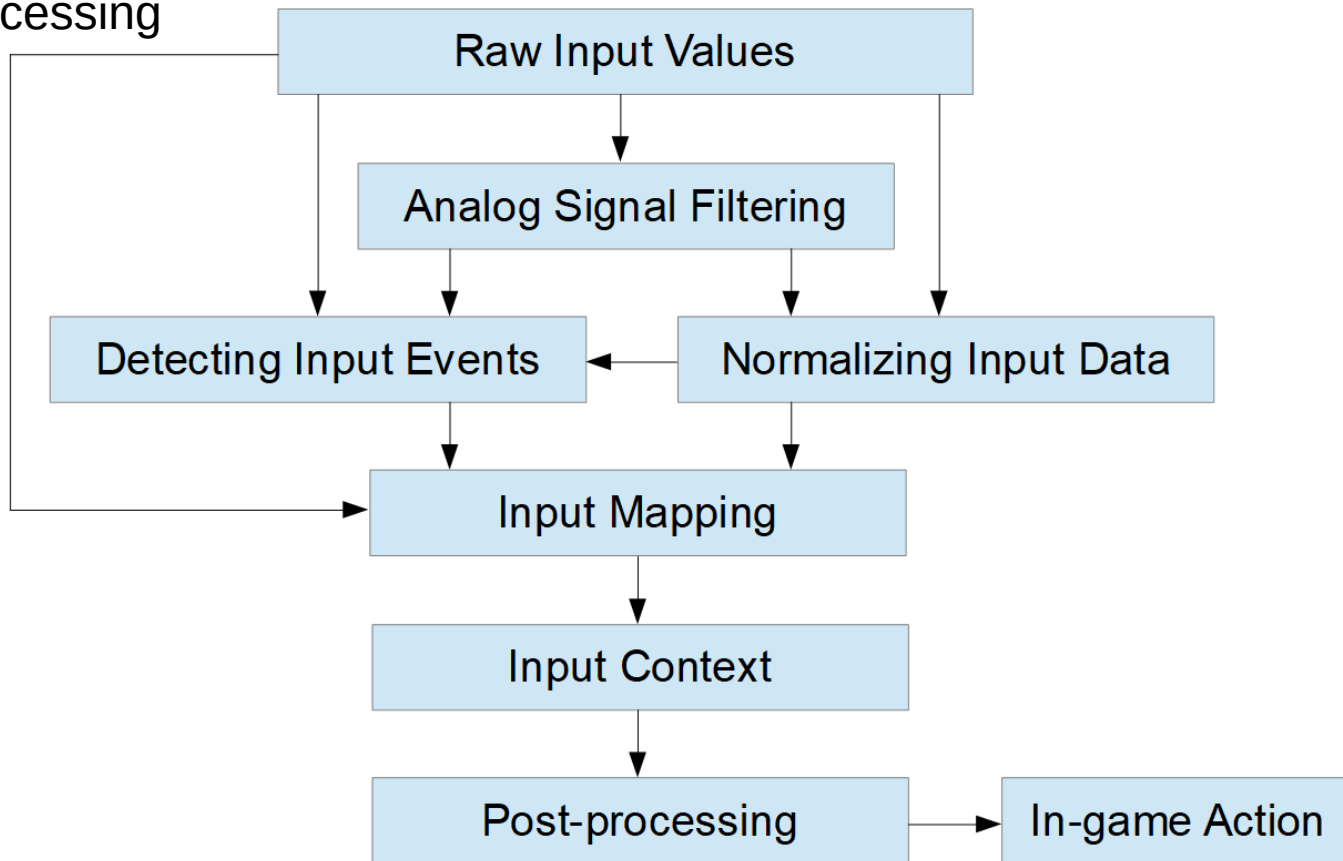


- A game engine's HID system usually provides some or all of the following features:
 - Dead zones,
 - Analog signal filtering,
 - Event detection (e.g., button up, button down),
 - Detection of button sequences and multi-button combinations (known as chords),
 - Gesture detection,
 - Management of multiple HID's for multiple players,
 - Multiplatform HID support,
 - Controller input re-mapping,
 - Context-sensitive inputs,
 - The ability to temporarily disable certain inputs.

Runtime Implementation (2/6)



- Input Processing



Runtime Implementation (3/6)



- Detecting Input Events
 - A **chord** is a group of buttons that, when pressed at the same time, produce a unique behavior in the game.
 - Watch the states of two or more buttons and only perform the requested operation when *all* of them are down.
 - A **gesture** is a sequence of actions performed via a HID by the player over a period of time.
 - Generally implemented by keeping a brief history of the HID actions performed by the player.
 - **Rapid Button Tapping**
 - We can detect the frequency of a button press by simply keeping track of the last time we saw a button-down event.
 - **Multibutton Sequence**
 - Keeping the sequence with an array of button ids.
 - **Thumb Stick Rotation**
 - Dividing the two-dimensional range of possible stick positions into quadrants.

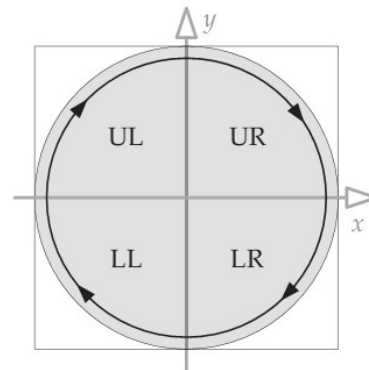


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

Runtime Implementation (4/6)



- Input Mapping
 - “Every problem in computer science can be solved with a level of indirection.”
(Professor Jay Black of the University of Waterloo)
 - A table maps each physical or abstract control index to a logical function in the game
 - Controls are grouped into a few classes
 - *Digital buttons*
 - All states packed into a 32-bit word, one bit per button
 - Or one floating-point value per button, which either is 0 or 1
 - *Unidirectional absolute axes* (e.g., triggers, analog buttons)
 - Produce floating-point input values in the range $[0, 1]$
 - *Bidirectional absolute axes* (e.g., joy sticks)
 - Produce floating-point input values in the range $[-1, 1]$
 - *Relative axes* (e.g., mouse axes, wheels, track balls)
 - Produce floating-point input values in the range $[-1, 1]$, where ± 1 represents the maximum relative offset possible within a single game frame

Runtime Implementation (5/6)



- Context-Sensitive Controls
 - A physical control can have different functions, depending on context, e.g.
 - The ubiquitous “use” button (pick up, open, talk, ...)
 - Modal control scheme (walking, driving, ...)
 - Determining the right context can be tricky, e.g.
 - Standing between a weapon and a health pack
 - Some games implement a priority system to break ties like this
 - Subtle context sensitivity, e.g.
 - Character walking in a direction relative to the camera angle

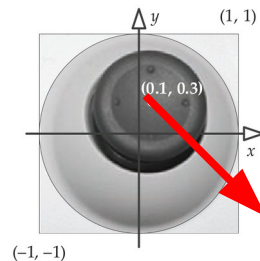
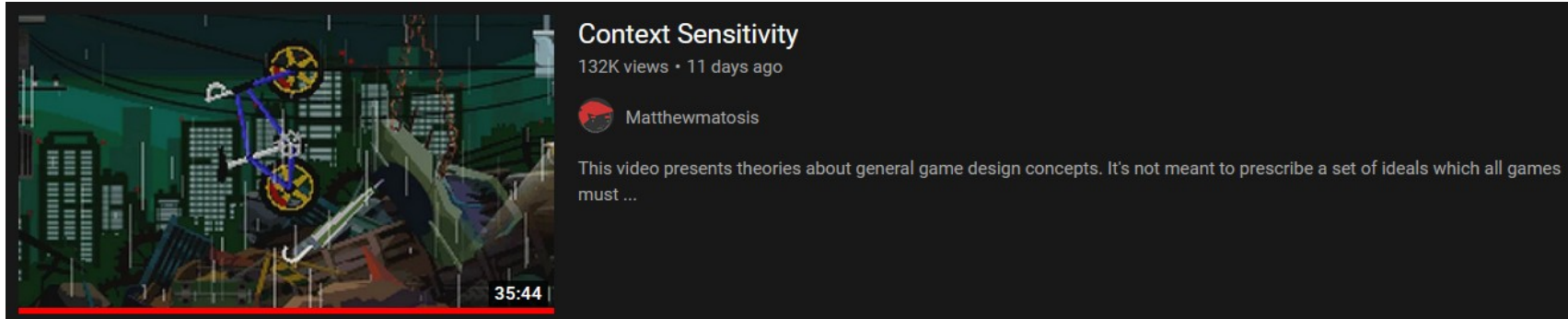


Image sources: <https://www.gta4.net/cheats/>, Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014, <https://www.xgn.nl/nieuws/75307/super-mario-galaxy-wii-u-versie-uit-in-amerika>

Runtime Implementation (6/6)



- Context Sensitivity of input in games



<https://www.youtube.com/watch?v=pl9G4voWfDQ>

References



- http://en.wikipedia.org/wiki/Human_interface_device
- http://en.wikipedia.org/wiki/PlayStation_Move
- <http://www.ign.com/boards/threads/new-wii-u-controller-details-magnetometer-and-more.204566379/>
- <https://play.google.com/store/apps/details?id=com.plaincode.magnetmeter&hl=en>
- <http://www.sensorplatforms.com/understanding-smart-phone-sensor-performance-magnetometer-2/>
- <https://www.ifixit.com/Teardown/Nintendo+Switch+Teardown/78263>
- <https://www.nintendo.com/switch/features/tech-specs/>
- <http://en.wikipedia.org/wiki/Touchscreen>
- http://en.wikipedia.org/wiki/Virtual_Boy